Hackathon Problem Statement - 1

1. Title of the Challenge - Smart Expense Management with Auto-Budgeting

As a college student or newly joined working professional or a seasoned professional of any sector we all make plans, we all spend in groups, we all spend across a variety of recurring necessities, and we do not keep a track. Tracking such expenses is laborious and time consuming, at the same time chances of error and getting right money back from the group members on recent trip or recent movie you watched together is a big pain because of multiple reasons.

But for all these problems and challenges in accessing/giving back the money to right person, you need to have an easy way to payout the money taken (e.g. Integration with existing PSP (Payment Service Providers)). On top of these budgeting your recurring expenses is one the major challenge these days.

2. Summary of the Problem Statement

- The goal is to create a Smart Expense Management that allows you create/delete/delegate your expenses in smarter way and Auto Budgeting system allows to categorize expenses, provides real-time insights, and dynamically adjusts budgets based on spending patterns and income changes.
- This solution with user consent aims to simplify financial management for better decision-making, and help individuals maintain control over their finances and adjusting budgets automatically. This will significantly improve finance management, reduce financial stress, and encourage better spending habits.
- The problem revolves around the difficulty of managing personal finances effectively due to manual tracking (no notification to collect or send money), inaccurate expense categorization, and static budgeting. Many individuals struggle with monitoring their spending habits and adjusting their budgets as expenses and income fluctuate.
- Auto budgeting systems analyze past spending, allows to put spend controls, gives deep insights in form of dashboards, avoid duplicate payments to same person for same contribution towards any group activity.

3. Must-Have Features (MVP)

(*These are essential features that the solution must include to be considered valid.*)

- Core functionality that solves the problem.

    1. Account management – Signing Up user have a profile setup based on user type (Student vs Earning professional)
    2. Smart way to capture expense
        - 2.1. Capture & categorising the expense
        - 2.2. Type of expense group created (travel, education course, shopping, dinning)
        - 2.3. Custom Categories: Allow users to create custom categories (e.g., "Gifts", "Pet Care", "Home Renovation") for specific expenses.
        - 2.4. Effective record keeping
    3. Easy to create and delete group
        - 3.1. Add/remove users
    4. Setting up notification and reminders
    5. Integrating with payment channel to complete the split payment
    6. Personal finance agent/bot to keep a track/behaviour of expenses
        - 6.1. Visual representations (graphs, charts, trend lines) to give users a clear picture of their spending patterns and financial health.
        - 6.2. Ways to eliminate duplicate payments
        - 6.3. Bot doing expense prediction for recurring bills that is getting split
    7. Giving Credit to your group members based on past success history
- **Basic user flow** (e.g., transaction initiation, authentication, processing).
- **Security measures** (e.g., encryption, fraud detection, basic compliance, data privacy).
- **Integration with existing payment systems/APIs**.

📌 **Example:**

✅ Notification for collect and pay with an auto-debit feature.

✅ A basic **dashboard** to track transactions.

4. Good-to-Have Features (Bonus Points for Innovation)

(*These are additional features that can improve usability, security, and efficiency but are not mandatory.*)

- Capturing or removing expense details via Voice/Image etc.
- **AI-powered recommendations**.
    - Virtual Assistant: An AI-powered chatbot or assistant that can answer finance-related questions, help users understand their spending habits, suggest budget adjustments, and even provide tips for improving their financial health.
    - Conversational Interface: Users can interact with the app via voice or text, asking about their current balance, upcoming bills, or potential savings tips.

- **Gamification or reward-based engagement**.
    - Rewards/points or coins earning by timely payment completion in the group; avoiding defaulter scenario.
    - Give rating to users on successful return of contribution, so that platform project these users as trusted and loyal to added in any group.
- **Cross-platform support** (web, mobile, wearable devices, android vs iOS).
- **Subscription Management:**
    - Subscription Tracker and Alerts: Automatically detect and categorize subscription services (e.g., Netflix, Spotify, gyms)

📌 **Example:**

⭐ Ability to **auto-sync** transactions when online or offline(cash mode).

⭐ **Multi-language support for voice based expense adding or deleting**.

5. Technology Stack & Integration

(*What technologies can be used? What existing APIs or services can be leveraged?*)

- **Recommended technologies**:
  - **Frontend**: React, Flutter (for mobile), or Angular.
  - **Backend**: Node.js, Django, or FastAPI.
  - **Database**: PostgreSQL, Firebase (for real-time sync), or MongoDB.
  - **AI/ML**: Python (TensorFlow, Scikit-Learn) for budget predictions.
  - **API Integration**: Plaid, Yodlee, or Finicity for banking data.
- **Possible integrations**:
  - Payment gateway APIs (e.g., Razorpay, Stripe, PayPal).
  - AI/ML tools (e.g., TensorFlow, OpenAI, AWS AI services).
  - Security frameworks (e.g., OAuth, JWT, biometric authentication).

📌 **Example:**

- Use **NFC + encrypted tokens** for offline payments.
- Store **transactions locally and sync** when online.
- Implement **secure authentication via biometrics or PINs**.

6. Scalability & Feasibility Aspects

(*How can this solution scale beyond the hackathon?*)

- **Will it work for millions of users?** If not, what optimizations are needed?
- **Can it handle peak transaction loads?**
- **What are the cost and infrastructure requirements?**
- **Can it be integrated with existing payment ecosystems easily?**

📌 **Example:**

- Solution should **support 100,000+ users** without major changes.
- Consider using **cloud-based infrastructure (AWS/GCP/Azure)** for scalability.
- Ensure low-cost implementation for **mass adoption**.

7. Target Audience & Impact

Identify your primary and secondary users for these solutions, considering the benefits and adoptability aspects.

- **Young Professionals & Graduates,** Early Adopters
- **Students:** Typically, on limited budgets, students can use auto budgeting to monitor spending and ensure they stay within their means.
    - Families & Households: With multiple income streams and varied expenses (childcare, mortgage, utilities), these users need comprehensive tools to manage diverse financial inflows and outflows.

Impact

- **Reduced Overspending:** Proactive alerts and personalized insights help users avoid common pitfalls like overspending, which can lead to debt accumulation.

- **Improved Savings:** By optimizing budgets and identifying unnecessary expenditures, users can allocate more funds towards savings and investments.

8. Estimated Cost to Build (Post-Hackathon Feasibility)

(*A rough estimate of cost factors for turning the hackathon project into a real-world product.*)

You may not need to include the actual cost of each component, but many to consider the options available wisely so that the optimum use of budget can be maintained.

Try to answer "**Why**" part for the tools and technologies selected.

- **Infrastructure costs** (servers, cloud storage, APIs).

- **Development costs** (team, software licenses, third-party services).

- **Compliance & security** (PCI-DSS, data privacy laws).

📌 **Example (for Offline Payments Solution):**
💰 **Development cost:** ~$5,000 for MVP
💰 **Cloud hosting & storage:** ~$500/month for 100,000 users
💰 **Compliance & security audits:** ~$10,000 for full-scale deployment

9. Judging Criteria

(*How will participants be evaluated? What are the key success metrics?*)

- **Innovation (30%)** – Uniqueness of the solution.
- **Technical Feasibility (25%)** – How well the technology is implemented.
- **Usability & UX (20%)** – Ease of use and user experience.
- **Scalability & Security (15%)** – Can the solution handle real-world usage?
- **Demo & Presentation (10%)** – Clarity of pitch and working prototype.

10. Deliverables at the End of Hackathon

(*What should participants submit?*)

- **A working prototype/demo.**
- **A brief pitch presentation (3-5 mins).**
- **A short technical write-up (1-2 pages) explaining architecture & technologies used.**
- **(Optional) Code repository (GitHub/Bitbucket) for evaluation.**